

NYKA BEAUTY STORE MANAGEMENT SYSTEM

PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM

1.Database Design

1.1 Review Table

Attribute	Key Type
Review_ID	Primary Key
Customer_ID	Foreign Key
Product_ID	Foreign Key
Rating	-
Review_Text	-
Review_Date	-

1.2 Rating Table

Attribute	Key Type
Rating_ID	Primary Key
Customer_ID	Foreign Key
Product_ID	Foreign Key
Rating_Value	-
Rating_Date	-

2.Table Relationships

Table	Related Table	Relationship
Customer	Review	One Customer can write many Reviews
Product	Review	One Product can have many Reviews

Table	Related Table	Relationship
Customer	Rating	One Customer can give many Ratings
Product	Rating	One Product can receive many Ratings

Relationship:

Customer → Review → Product

Customer → Rating → Product

3. Table Creation

3.1 Create Review Table

```
SQL> CREATE TABLE Review (
  2     Review_ID NUMBER PRIMARY KEY,
  3     Customer_ID NUMBER,
  4     Product_ID NUMBER,
  5     Rating NUMBER(1),
  6     Review_Text VARCHAR2(500),
  7     Review_Date DATE
  8 );

Table created.
```

3.2 Create Rating Table

```
SQL> CREATE TABLE Rating (
  2     Rating_ID NUMBER PRIMARY KEY,
  3     Customer_ID NUMBER,
  4     Product_ID NUMBER,
  5     Rating_Value NUMBER(1),
  6     Rating_Date DATE
  7 );

Table created.
```

4. Review Data Insertion

```
SQL> INSERT INTO Review VALUES
  2 (1, 101, 201, 5, 'Excellent product', DATE '2026-09-01');

1 row created.

SQL>
SQL> INSERT INTO Review VALUES
  2 (2, 102, 202, 4, 'Very good quality', DATE '2026-09-02');

1 row created.

SQL>
SQL> INSERT INTO Review VALUES
  2 (3, 103, 203, 3, 'Average product', DATE '2026-09-03');

1 row created.

SQL>
SQL> INSERT INTO Review VALUES
  2 (4, 104, 201, 5, 'Very useful product', DATE '2026-09-04');

1 row created.

SQL>
SQL> INSERT INTO Review VALUES
  2 (5, 105, 204, 4, 'Good product', DATE '2026-09-05');

1 row created.

SQL>
SQL> COMMIT;

Commit complete.
```

5.Rating Data Insertion

```
SQL> INSERT INTO Rating VALUES
  2 (1, 101, 201, 5, DATE '2026-09-01');

1 row created.

SQL>
SQL> INSERT INTO Rating VALUES
  2 (2, 102, 202, 4, DATE '2026-09-02');

1 row created.

SQL>
SQL> INSERT INTO Rating VALUES
  2 (3, 103, 203, 3, DATE '2026-09-03');

1 row created.

SQL>
SQL> INSERT INTO Rating VALUES
  2 (4, 104, 201, 5, DATE '2026-09-04');

1 row created.

SQL>
SQL> INSERT INTO Rating VALUES
  2 (5, 105, 204, 4, DATE '2026-09-05');

1 row created.

SQL>
SQL> COMMIT;

Commit complete.
```

6.Retrieve Product Review Details

```
SQL> SELECT
2     Review_ID,
3     Customer_ID,
4     Product_ID,
5     Rating,
6     Review_Text,
7     Review_Date
8 FROM Review
9 ORDER BY Review_ID;
```

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID	RATING
-----------	-------------	------------	--------

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID	RATING
-----------	-------------	------------	--------

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID	RATING
-----------	-------------	------------	--------

1	101	201	5
---	-----	-----	---

Excellent product
01-SEP-26

2	102	202	4
---	-----	-----	---

Very good quality
02-SEP-26

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID	RATING
-----------	-------------	------------	--------

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID	RATING
-----------	-------------	------------	--------

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID	RATING
-----------	-------------	------------	--------

3	103	203	3
---	-----	-----	---

Average product
03-SEP-26

7. Calculate Average Product Ratings

```
SQL> SELECT
2     Product_ID,
3     AVG(Rating_Value) AS Average_Rating
4 FROM Rating
5 GROUP BY Product_ID;
```

PRODUCT_ID	AVERAGE_RATING
------------	----------------

201	5
202	4
203	3
204	4

8. Identify Highly Rated Products

```
SQL> SELECT
  2     Product_ID,
  3     AVG(Rating_Value) AS Average_Rating
  4 FROM Rating
  5 GROUP BY Product_ID
  6 HAVING AVG(Rating_Value) >= 4;
```

PRODUCT_ID	AVERAGE_RATING
201	5
202	4
204	4

9.Product Rating Report

```
SQL> SELECT
  2     r.Product_ID,
  3     COUNT(rt.Rating_ID) AS Total_Ratings,
  4     AVG(rt.Rating_Value) AS Average_Rating,
  5     MAX(rt.Rating_Value) AS Highest_Rating,
  6     MIN(rt.Rating_Value) AS Lowest_Rating
  7 FROM Review r
  8 JOIN Rating rt
  9 ON r.Product_ID = rt.Product_ID
 10 GROUP BY r.Product_ID
 11 ORDER BY Average_Rating DESC;
```

PRODUCT_ID	TOTAL_RATINGS	AVERAGE_RATING	HIGHEST_RATING	LOWEST_RATING
201	4	5	5	5
204	1	4	4	4
202	1	4	4	4
203	1	3	3	3